

FSP

Secure Coding Practices

Secure Coding Practices for Android

- With the Android platform fast becoming a target of malicious hackers, applications security is no longer an add-on, but a crucial part of the developer's job.
- This session provides secure coding practices that a developer need to follow in order to design and implement robust, rugged, and secure apps for Android device.



**Verify for Security Early
and Often 01**

04 Validate All Inputs

Parameterized Queries 02

**05 Implement Identity and
Authentication Controls**

Encode Data 03

**06 Cryptographically
Secure Data**

**Implement Logging and
Intrusion Detection**

07

09

**Monitor Error and
Exception Handling**

**Leverage Security
Frameworks and
Libraries**

08

10

Handling Intents

11

SSL Pinning

01

Verify for Security Early and Often

Securing Application Components :

- Application Components are the essential building blocks of an Android Application.
- Each component is an entry point through which the system or a user can enter your application.

Securing Application Components :

- Four major Android Application Components are:
 - Activities
 - Services
 - Content Providers
 - Broadcast Receivers

Securing Application Components :

- One of the common mistakes while developing applications is unintentionally leaving application components exposed.
- Application components can be secured both by making proper use of the **AndroidManifest.xml** file and by forcing permission checks at code level.
- These two factors of application security make the permissions framework quite flexible and allow you to limit the number of applications accessing your components in quite a granular way.

Securing Application Components :

- The **android:exported** attribute defines whether a component can be invoked by other applications.
- If any of your application components do not need to be invoked by other applications or need to be explicitly shielded from interaction with the components on the rest of the Android system (other than components internal to your application)

Securing Application Components :

- You should add the following attribute to the application component's XML element:



```

<[component name] android:exported="false">
</[component name]>
  
```

- Here the [component name] would either be an activity, provider, service, or receiver.

Protecting components with Custom Permissions :

- The Android platform defines a set of default permissions, which are used to secure system services and application components.
- Largely, these permissions work in the most generic case, but often when sharing bespoke functionality or components between applications it will require a more tailored use of the permissions framework.
- **This is facilitated by defining custom permissions.**

Following snippets demonstrates how you can define your own custom permissions:

1. Before adding any custom permissions, you need to declare string resources for the permission labels. You can do this by editing the strings.xml file in your application project folder under **res/values/strings.xml**:



```
<string name="custom_permission_label">Custom Permission</string>.
```

- Adding normal protection-level custom permissions to your application can be done by adding the following lines to your **AndroidManifest.xml** file:



```

<permission android:name="android.permission.CUSTOM_PERMISSION"
  android:protectionLevel="normal"
  android:description="My custom permission"
  android:label="@string/custom_permission_label">
  
```

3. Making use of this permission works the same as any other permission; you need to add it to the **android:permission** attribute of an application component.



```

<[component name] ...
    android:permission="android.permission.CUSTOM_PERMISSION">
</[component name]>
  
```

- Here the [component name] would either be an activity, provider, service, or receiver.

- You can also allow other applications to request this permission by adding the **<uses-permission/>** tag to an application's AndroidManifest.xml file:



```
<uses-permission android:name="android.permission.CUSTOM_PERMISSION"/>
```

The breakdown of the attributes of `<permission>` element is as follows:

- **android:name** – This defines the name of the permissions, which is the string value that will be used to reference this permission.
- **android:protectionLevel** – This defines the protection level of the permission and controls whether users will be prompted to grant the permission.

Following are the Protection Levels:

- **normal** – This permission is used to define non dangerous permissions, these permissions will not be prompted and may be granted autonomously.
- **dangerous** – This permission is used to define permissions that expose the user to considerable fiscal, reputational, and legal risk.

Following are the Protection Levels:

- **signature** – This permission is granted autonomously to applications that are signed with the same key as the application that defines them.
- **signatureOrSystem** – This permission is automatically granted to any application that forms a part of the system image or is signed with the same key as the application that defines them.

02

Parameterized Queries

SQL Injection :

- One specific form of a class of vulnerabilities known as command injection.
- In this type of vulnerability, input from an outside source is used directly as part of a command, in this case a SQL expression that is passed to a database interpreter.

For Example:

- consider a case where the user supplies a username and password to an application, which must then verify that the combination is valid to allow access.



```

1 String loginQuery = "SELECT * FROM useraccounts WHERE userID = '" +
2   request.getParameter("userID") + "' AND password = '" +
3   request.getParameter("password") + "'";
  
```

For Example:

- However, if whoever is submitting the information were to submit these values:



```

1
2     userID = ' or 1=1 --
3     password = doesNotMatter
4
5
  
```

For Example:

- The resulting query that would be supplied to the SQL database interpreter would be:

```
1 SELECT * FROM useraccounts WHERE userID = '' or 1=1 -- AND password='doesNotMatter'
```

For Example:

- This SQL statement would then evaluate and return all of the data from the user accounts table, as the WHERE condition would always be true (due to the OR 1=1 condition) and the password checking clause would be commented out.

```
1 SELECT * FROM useraccounts WHERE userID = '' or 1=1 -- AND password='doesNotMatter'
```

This unintended behavior is made possible because of two primary problems.

- **First**, our app did not properly validate the input that was received before using that input to form the SQL statement to be submitted to the database interpreter.
- **Second**, the problem is enabled because when query strings are constructed in such a manner, the database system cannot tell the difference between code and data; the initial apostrophe (') submitted in the userID is actually data, but is being interpreted as code because the database cannot tell that it should be interpreted as a literal data value.

Preventing Command Injection :

- Looking at an example similar to the earlier one.
- Let's consider looking up a user's last name in a database. The unsafe way to form this statement looks something like this:



```

1 SQLiteDatabase db = dbHelper.getWritableDatabase();
2 String userQuery = "SELECT lastName FROM useraccounts WHERE userID = " +
3     request.getParameter("userID");
4 SQLiteStatement preparedStatement = db.compileStatement(userQuery);
5 String userLastName = preparedStatement.simpleQueryForString();
  
```

Preventing Command Injection :

- Here is the proper way to perform such a query against the database, where the command is separated from the data:
- (compliant code)



```
1 SQLiteDatabase db = dbHelper.getWritableDatabase();
2 String userQuery = "SELECT lastName FROM useraccounts WHERE userID = ?";
3 SQLiteStatement prepStatement = db.compileStatement(userQuery);
4 prepStatement.bindString(1, request.getParameter("userID"));
5 String userLastName = prepStatement.simpleQueryForString();
```

Preventing Command Injection :

- By taking advantage of the compileStatement capability, we can effectively separate commands from data in SQL statements, by using the `?` marker.
- This is known as a parameterized query, as the query string includes placeholders (question marks) to mark the data and the values for those pieces of data are filled in independently (by the `bindString()` call in our example).

03

Encode Data

- Encoding your data is a solution you should consider if you work with slightly less sensitive data or are looking for a way to organize your data.
- Most encoding methods rely on algorithms to compress the data and reduce its complexity.
- The same algorithm used to encode the data is needed to access the data in a readable format.

- Encoding keeps your data safe since the data is not readable unless you have access to the algorithms that were used to encode it.
- This is a good way to protect your data from theft since any stolen data would not be usable.
- Encoding is an ideal solution if you need to have third parties access your data but do not want to have everyone be able to access some sensitive data.

- Since encoding removes redundancies from data, the size of your data will be a lot smaller.
- This results in faster input speed when data is processed or saved.
- Since encoded data is smaller in size, you should be able to save space on your storage devices.
- Encoded data is easy to organize, even if the original data was mostly unstructured.

- This is how we can encode a normal string to a Base64 encoding scheme.
- (compliant code)



```
1 String testValue = "Hello, world!";
2
3 byte[] encodeValue = Base64.encode(testValue.getBytes(), Base64.DEFAULT);
4 byte[] decodeValue = Base64.decode(encodeValue, Base64.DEFAULT);
5
6 Log.d("ENCODE_DECODE", "defaultValue = " + testValue);
7 Log.d("ENCODE_DECODE", "encodeValue = " + new String(encodeValue));
8 Log.d("ENCODE_DECODE", "decodeValue = " + new String(decodeValue));
```

- The output will be like the following when encoding with Base64 scheme.
- (compliant code)



```

1 //OUTPUT
2 defaultValue = Hello, world!
3 encodeValue = SGVsbG8sIHdvcmxkIQ==
4 decodeValue = Hello, world!
  
```

04

Validate User Input

- Input validation is the key component of application security.
- **One of the primary problems in applications is that developers trust that input.**
- Remember that any piece of data submitted from outside your app can be submitted by, or manipulated by, an attacker.
- You need to verify that the data received by your application is valid and safe to process.

- Validating input data is the easiest and yet most effective secure coding method.
- All data that is inputted into the application either directly or indirectly by an outside source needs to be properly validated.

For Example:

- Following is an example of an Activity as used in a program that receives data from Intent:

```

1 TextView tv = (TextView) findViewById(R.id.textview);
2 InputStreamReader isr = null;
3 char[] text = new char[1024];
4 int read;
5 try {
6     String urlstr = getIntent().getStringExtra("WEBPAGE_URL");
7     URL url = new URL(urlstr);
8     isr = new InputStreamReader(url.openConnection().getInputStream());
9     while ((read=isr.read(text)) != -1) {
10         tv.append(new String(text, 0, read));
11     }
12 } catch (MalformedURLException e) { //...
  
```

- The example is a simple sample where HTML is acquired from a remote web page in a designated URL and the code is displayed in TextView.
- However, this is not sufficient.
- Furthermore, when a **"file://..."** formatted URL is designated by urlstr, the file of the internal file system is opened and is displayed in TextView rather than the remote web page.

- The below example shows a revision to fix the security bugs:

```

1 TextView tv = (TextView) findViewById(R.id.textview);
2 InputStreamReader isr = null;
3 char[] text = new char[1024];
4 int read;
5 try {
6     String urlstr = getIntent().getStringExtra("WEBPAGE_URL");
7     URL url = new URL(urlstr);
8     String prot = url.getProtocol();
9     if (!"http".equals(prot) && !"https".equals(prot)) {
10         throw new MalformedURLException("invalid protocol");
11     }
12     isr = new InputStreamReader(url.openConnection().getInputStream());
13     while ((read=isr.read(text)) != -1) {
14         tv.append(new String(text, 0, read));
15     }
16 } catch (MalformedURLException e) { //...
  
```

- Validating the safety of input data is called **"Input Validation"** and it is a fundamental secure coding method.

05

Implement Identity and Authentication Controls

- Providing a secure login mechanism for your users is harder than on the Web.
- The trend on mobile devices is to make things as easy as possible for the user.
- But if you make it too easy to login into your app, you run the risk of unauthorized users gaining access to sensitive data by going around this authentication.

- The following tokens are common on Android devices as a part of login process:
 - Username and Password
 - Device information, such as DeviceID and AndroidID
 - Network information, such as IP address
- The classic login of username and password is still the most common authentication on an Android phone.

- Let's look at some best practices for user authentication.
- The best practices are as follows:
 - No password caching
 - Minimum password length
 - Multi-factor authentication
 - Server-side as well as client-side authentication

- Do not save or cache username, and especially password, information on the phone, as there is always a risk that it will be found and decrypted.
- Even if you're encrypting the password, if you're also storing the key in the APK then it's going to be unencrypted.
- It is better not to store passwords, if you can get away with it, and make the user Log In each time.

- **Try to enforce a minimum password length** – passwords of less than six characters are highly prone to a brute force attack. Financial apps should have stricter policies than other apps.
- **Validate email addresses** – this can be done either using regular expressions or via an email link during setup or, better still, using both approaches.
- If you do update your password standards, notify your existing customers when they login again to update their passwords.

- It's becoming very common for applications to use a **Two-Factor Authentication** where a randomly generated PIN number is sent via SMS message to user's phone before he can log in to the application.
- We can also use the info like DeviceID, IP Address, and Location information to add extra layers of information.

- **Access control** doesn't end at the client; it needs to be enforced at the server, too.
- Some back-end servers mistakenly rely on the client app to perform all authentication and assume that the web server doesn't need to do any authentication.
- The server should also check for valid credentials each time or use a session token once again over SSL.
- It should also check for unusual activity, such as someone performing a bruteforce attack, and notify the user via email of unusual login activity.

- If you are saving any personal, healthcare, or financial information, you should use an **asymmetric** or **public/private** key.
- This does require a round trip back to the server to decrypt the data, but if the phone is compromised, the user's data will remain secure.
- Only the private key can decrypt the data, and that should never be stored on the phone.

5.1

Network Security for Android Applications

- Network security for Android applications protects data exchanged between the app and servers from unauthorized access and tampering.
- It is relevant because mobile apps handle sensitive user information, and securing network communication prevents attacks like Man-in-the-Middle (MitM) and data breaches.
- It is necessary to ensure data integrity, confidentiality, and authenticity, while building user trust and complying with security regulations.

Essential Network Security Techniques:

- **Always use HTTPS:** Communicate with servers only over HTTPS to ensure data encryption and integrity.
- **Implement data encryption:** Encrypt sensitive data like user credentials and financial information both in transit and at rest.
- **Validate server certificates:** Verify server certificates using a trusted certificate authority to prevent MitM attacks.
- **Utilize network security libraries:** Leverage libraries like OkHttp and Retrofit for secure network communication and built-in security features.
- **Perform vulnerability scanning:** Regularly scan your app and its dependencies for vulnerabilities related to network communication.
- **Monitor network activity:** Monitor network activity to detect suspicious behavior and potential security threats.

Implementing Network Security:

- Configure your app to use HTTPS by default.
- Utilize libraries like OkHttp and Retrofit for secure network communication.
- Implement certificate pinning to verify server certificates and prevent MitM attacks.
- Validate user input to prevent injection attacks and unauthorized data modification.
- Securely store sensitive data like encryption keys and access tokens using the Android Keystore.
- Monitor network activity and log security events to detect suspicious behavior.

Network security configuration:

- The Network Security Config feature lets apps customise their network security settings in a safe, declarative configuration file without modifying app code.
- These settings can be configured for specific domains and for a specific app.

Network security configuration:

The key capabilities of this feature are as follows:

- **Custom trust anchors:** Customize which Certificate Authorities (CA) are trusted for an app's secure connections. For example, trusting particular self-signed certificates or restricting the set of public CAs that the app trusts.
- **Debug-only overrides:** Safely debug secure connections in an app without added risk to the installed base.
- **Cleartext traffic opt-out:** Protect apps from accidental usage of cleartext traffic.
- **Certificate pinning:** Restrict an app's secure connection to particular certificates.

Add a Network Security Configuration file:

The Network Security Configuration feature uses an XML file where you specify the settings for your app.

Create a **network_security_config.xml** file in the res/xml directory.

```
1 <network-security-config>
2   <domain-config cleartextTrafficPermitted="false">
3     <domain includeSubdomains="true">yourdomain.com</domain>
4   </domain-config>
5 </network-security-config>
```

Reference it in **AndroidManifest.xml**.

```
1 <application
2   android:networkSecurityConfig="@xml/network_security_config">
```

Customize trusted CAs (Certificate Authorities)

By default, secure connections (using protocols like TLS and HTTPS) from all apps trust the pre-installed system CAs, and apps targeting Android 6.0 (API level 23) and lower also trust the user-added CA store by default.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <network-security-config>
3   <domain-config>
4     <domain includeSubdomains="true">example.com</domain>
5     <domain includeSubdomains="true">secure.example.com</domain>
6     <trust-anchors>
7       <certificates src="@raw/my_ca"/>
8       <certificates src="system"/>
9     </trust-anchors>
10  </domain-config>
11 </network-security-config>

```

Configure CAs for debugging:

When debugging an app that connects over HTTPS, you may want to connect to a local development server, which does not have the SSL certificate for your production server.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <network-security-config>
3   <debug-overrides>
4     <trust-anchors>
5       <certificates src="@raw/debug_cas"/>
6     </trust-anchors>
7   </debug-overrides>
8 </network-security-config>
  
```

In order to support this without any modification to your app's code, you can specify debug-only CAs, which are trusted only when `android:debuggable` is true, by using **debug-overrides**.

Opt out of cleartext traffic:

Applications intending to connect to destinations using only secure connections can opt-out of supporting cleartext (using the unencrypted HTTP protocol instead of HTTPS) to those destinations.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <network-security-config>
3   <domain-config cleartextTrafficPermitted="false">
4     <domain includeSubdomains="true">secure.example.com</domain>
5   </domain-config>
6 </network-security-config>
  
```

For example, an app may want to ensure that all connections to secure.example.com are always done over HTTPS to protect sensitive traffic from hostile networks.

Pin certificates:

Certificate pinning is done by providing a set of certificates by hash of the public key (SubjectPublicKeyInfo of the X.509 certificate). A certificate chain is then valid only if the certificate chain contains at least one of the pinned public keys.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <network-security-config>
3   <domain-config>
4     <domain includeSubdomains="true">example.com</domain>
5     <pin-set expiration="2018-01-01">
6       <pin digest="SHA-256">7HIpactkIAq2Y49orF00QKurWxmmSFZhBCoQYcRhJ3Y=</pin>
7       <!-- backup pin -->
8       <pin digest="SHA-256">fwza0LRMXouZHRc8Ei+4PyulDPDcf3UKg0/04cDM1oE=</pin>
9     </pin-set>
10  </domain-config>
11 </network-security-config>
  
```

06

Cryptographically Secure Data

- Encryption techniques are frequently used to ensure confidentiality and integrity, and Android is equipped with a variety of cryptographic features to allow applications to realize confidentiality and integrity.
- You may use password-based key encryption for the purpose of protecting a user's confidential data assets.

Key Points to consider while Encrypting and Decrypting with Password-based keys:

- Explicitly specify the encryption mode and the padding.
- Use strong encryption technologies including algorithms, block cipher modes, and padding modes.
- When generating a key from password, use Salt.
- When generating a key from password, specify an appropriate hash iteration count.
- Use a key of length sufficient to guarantee the strength of encryption.

Here is the complete listing that will allow the encryption or decryption of data based on a password:

```

1 String password = ...;
2
3 String PBE_ALGORITHM = "PBKWithSHA256And256BitAES-CBC-BC";
4 String CIPHER_ALGORITHM = "AES/CBC/PKCS5Padding";
5 int NUM_OF_ITERATIONS = 1000;
6 int KEY_SIZE = 256;
7
8 byte[] salt = new SecureRandom().nextBytes(new byte[8]);
9 byte[] iv = new SecureRandom().nextBytes(new byte[16]);
10
11 String clearText = ...; // This is the value to be encrypted.
12 byte[] encryptedText;
13 byte[] decryptedText;
14 try
15 {
16     PBEKeySpec pbeKeySpec = new PBEKeySpec(password.toCharArray(),
17     salt, NUM_OF_ITERATIONS, KEY_SIZE);
18     SecretKeyFactory keyFactory = SecretKeyFactory.getInstance(PBE_ALGORITHM);
19     SecretKey tempKey = keyFactory.generateSecret(pbeKeySpec);
20     SecretKey secretKey = new SecretKeySpec(tempKey.getEncoded(), "AES");
21     IvParameterSpec ivSpec = new IvParameterSpec(iv);
22     Cipher encCipher = Cipher.getInstance(CIPHER_ALGORITHM);
23     encCipher.init(Cipher.ENCRYPT_MODE, secretKey, ivSpec);
24     Cipher decCipher = Cipher.getInstance(CIPHER_ALGORITHM);
25     decCipher.init(Cipher.DECRYPT_MODE, secretKey, ivSpec);
26     encryptedText = encCipher.doFinal(clearText.getBytes());
27     decryptedText = decCipher.doFinal(encryptedText);
28     String sameAsClearText = new String(decryptedText);
29 }
30 catch (Exception e)
31 {
32     ...
33 }

```

- The below section of code just sets up the algorithms and parameters we are going to use.
- Here we are using **SHA-256** as the Hashing algorithm and **AES** in **CBC** mode as the Encryption Algorithm.
- We will use 1,000 iterations of the hashing function during the process and will end up with a 256-bit key.



```

1 String PBE_ALGORITHM = "PBESWithSHA256And256BitAES-CBC-BC";
2 String CIPHER_ALGORITHM = "AES/CBC/PKCS5Padding";
3 int NUM_OF_ITERATIONS = 1000;
4 int KEY_SIZE = 256;
  
```

- We will be using AES, still in Cipher Block Chaining mode, and employ **PKCS#5** padding. The padding setting specified how to pad the data being encrypted (or decrypted) if it is not in multiples of 128 bits, as AES operates on blocks of 128 bits at a time.
- These all are standard algorithms and parameters, representing a strong encryption scheme.



```

1 String PBE_ALGORITHM = "PBESWithSHA256And256BitAES-CBC-BC";
2 String CIPHER_ALGORITHM = "AES/CBC/PKCS5Padding";
3 int NUM_OF_ITERATIONS = 1000;
4 int KEY_SIZE = 256;
  
```

- Here, we set up the **Salt** that will be used in the key derivation computation and the **Initialization Vector (IV)** that will be used in encryption and decryption of data.



```

1 byte[] salt = new SecureRandom.nextBytes(new byte[8]);
2 byte[] iv = new SecureRandom.nextBytes(new byte[16]);
  
```

- Here, we just set up the values we will encrypt and decrypt.
- For the purpose of this example, **clearText** will hold the string we want to encrypt.
- **encryptedText** is a byte array that will hold the encrypted value of the string.
- **decryptedText** will hold the decrypted bytes, once we encrypt the string and then decrypt it.



```
1 String clearText = ...; // This is the value to be encrypted.  
2 byte[] encryptedText;  
3 byte[] decryptedText;
```

- Here, we perform key derivation, going from the user-supplied password to a 256-bit AES key.



```

1 PBEKeySpec pbeKeySpec = new PBEKeySpec(password.toCharArray(),
2   salt, NUM_OF_ITERATIONS, KEY_SIZE);
3 SecretKeyFactory keyFactory = SecretKeyFactory.getInstance(PBE_ALGORITHM);
4 SecretKey tempKey = keyFactory.generateSecret(pbeKeySpec);
5 SecretKey secretKey = new SecretKeySpec(tempKey.getEncoded(), "AES");
  
```

- We then set up the Initialization Vector (IV) like this:



```
1 IvParameterSpec ivSpec = new IvParameterSpec(iv);
```

- This is the main section of our example. We use the derived key and IV, along with our specification of what encryption parameters we want to use, to form a Cipher object.
- We actually create two, one in **ENCRYPT_MODE** and one in **DECRYPT_MODE**.
- Once we have the cipher objects, we can perform encryption and decryption operations.



```

1 Cipher encCipher = Cipher.getInstance(CIPHER_ALGORITHM);
2 encCipher.init(Cipher.ENCRYPT_MODE, secretKey, ivSpec);
3
4 Cipher decCipher = Cipher.getInstance(CIPHER_ALGORITHM);
5 decCipher.init(Cipher.DECRYPT_MODE, secretKey, ivSpec);
  
```

- For encryption, we pass the string we want to encrypt (convert into a byte array) to the **doFinal()** method of the encrypting Cipher object.
- The encryption operation is performed and the resulting encrypted bytes are returned to us.



```
1 encryptedText = encCipher.doFinal(clearText.getBytes());
```

- We can also decrypt data using the same approach. In this case, we use the decrypting Cipher object, passing it the encrypted bytes and getting back to plaintext bytes.
- As we started a string, we can reconstruct it, and the **sameAsClearText** string will contain exactly the same value as the **clearText** string.



```

1 decryptedText = decCipher.doFinal(encryptedText);
2 String sameAsClearText = new String(decryptedText);
  
```

- And that's it. We now have a complete solution for protecting stored data, where we take a user-supplied password, derive a symmetric key from it, and encrypt and decrypt data at will, all in a few short lines of code.
- Once we encrypt the data, we can store it in a file, in a database, or wherever. As long as we keep the symmetric key from compromise, an attacker will not be able to recover the data from its encrypted form.

07

Implement Logging and Intrusion Detection

Tamper Detection:

- Tamper or Intrusion detection is the ability of an application to sense that an active attempt to compromise the application's integrity or the data associated with the application is in progress; the detection of the threat may enable the application to initiate appropriate defensive actions.

Responding to Tamper Detection:

- The obvious and simple solution would be to check for tampering on startup, and if detected, exit the app optionally with a message to the user explaining why.
- In the next slides, we will look at different checks that may indicate a tampered, compromised, or hostile environment.
- These are designed to be activated once you are ready for release.

Detect if Google Play store was the installer:

- Detecting if the installer was the Google Play store is a simple check that the package name of the installer app matches that of the Google Play store.
- Specifically, it checks if the installer package starts with **com.google.android**.
- It is a useful check if you are distributed solely through the Google store.

Detect if Google Play store was the installer:



```
1 public static boolean checkGooglePlayStore(Context context) {  
2     String installerPackageName = context.getPackageManager().  
3         getInstallerPackageName(context.getPackageName());  
4  
5     return installerPackageName != null && installerPackageName.startsWith("com.google.android");  
6 }
```

Detect if it runs on an Emulator:

- The Java Reflection API makes it possible to inspect classes, methods, and fields at runtime; and in this case, allows us to override the access modifiers that would prevent ordinary code from compiling.
- The emulator check uses reflection to access a hidden system class, `android.os.SystemProperties`.
- **NOTE:** using hidden APIs can be risky, as they can change between Android versions.

Detect if it runs on an Emulator:



```

1 public static boolean isEmulator() {
2     try {
3         Class systemPropertyClazz = Class.forName("android.os.SystemProperties");
4         boolean kernelQemu = getProperty(systemPropertyClazz, "ro.kernel.qemu").length() > 0;
5         boolean hardwareGoldfish = getProperty(systemPropertyClazz, "ro.hardware").equals("goldfish");
6         boolean modelSdk = getProperty(systemPropertyClazz, "ro.product.model").equals("sdk");
7         if (kernelQemu || hardwareGoldfish || modelSdk) {
8             return true;
9         }
10    } catch (Exception e) {
11        // error assumes emulator
12    }
13    return false;
14 }
  
```

Detect if it runs on a Rooted Device:



```

1 public static boolean checkRoot(){
2     for(String pathDir : System.getenv("PATH").split(":")){
3         if(new File(pathDir, "su").exists()) {
4             return true;
5         }
6     }
7     return false;
8 }
  
```

Detect if the app has the “debuggable” flag enabled: (something that should only be enabled during development)

- When debuggable is enabled, it is possible to connect via the Android Debug Bridge and perform detailed dynamic analysis.
- The debuggable variable is a simple property of the **<application>** element in the **AndroidManifest.xml** file.
- It is perhaps one of the easiest and most targeted properties to alter in order to perform dynamic analysis.

Detect if the app has the “debuggable” flag enabled:



```
1 public static boolean isDebuggable(Context context) {  
2     return (context.getApplicationInfo().flags & ApplicationInfo.FLAG_DEBUGGABLE) != 0;  
3 }
```

Application Signature Verification:

- Part of the process of an attacker modifying your application's **.apk** file breaks the digital signature.
- This means that, if they want to install the **.apk** file on an Android device, it will need to be resigned using a different signing key.
- Fortunately, at runtime, Android apps can query **PackageManager** to find app signatures.

Application Signature Verification:

```

1 // CERTIFICATE'S SHA1 Signature
2 private static String CERTIFICATE_SHA1 = "71920AC9486E087DCBCF5C7F6FEC95213585BCC5";
3
4 public static boolean validateAppSignature(Context context) {
5     try {
6         // get the signature form the package manager
7         PackageInfo packageInfo = context.getPackageManager().getPackageInfo(context.getPackageName(),
8             PackageManager.GET_SIGNATURES);
9
10        Signature[] appSignatures = packageInfo.signatures;
11
12        //this sample only checks the first certificate
13        for (Signature signature : appSignatures) {
14            byte[] signatureBytes = signature.toByteArray();
15            //calc sha1 in hex
16            String currentSignature = calcSHA1(signatureBytes);
17            //compare signatures
18            return CERTIFICATE_SHA1.equalsIgnoreCase(currentSignature);
19        }
20    } catch (Exception e) {
21        // if error assume failed to validate
22    }
23    return false;
24 }

```

Outputting Log to LogCat:

- There's a logging mechanism called **LogCat** in Android, and not only system log information but also application log information outputs to the Logcat.
- Log information in LogCat can be read out from other application in the same device.
- So the application which outputs sensitive information to LogCat, is considered that it has the vulnerability of the information leakage.

Outputting Log to LogCat:

- From a security point of view, in release version application, it's preferable that any log should not be output.
- In next slides, we will look some ways to output messages to LogCat in a safe manner even in a release version application.

Outputting Log to LogCat:

- **ProGuard** – the method to control the Log output in release version application.
- **ProGuard** is one of the optimization tools which automatically delete the unnecessary code like *unused methods*, etc.

Outputting Log to LogCat:

- Essentially there are five types of Log output methods:
 - **Log.e()** `//ERROR`
 - **Log.w()** `//WARN`
 - **Log.i()** `//INFO`
 - **Log.d()** `//DEBUG`
 - **Log.v()** `//VERBOSE`

Outputting Log to LogCat:

- It's recommended to use the following methods for outputting **operation** log information:
 - **Log.e()** `//ERROR`
 - **Log.w()** `//WARN`
 - **Log.i()** `//INFO`
- And the following for outputting **development** log information:
 - **Log.d()** `//DEBUG`
 - **Log.v()** `//VERBOSE`

Outputting Log to LogCat:

- The following code snippet from an application includes **Log.d()** and **Log.v()** for outputting debug log.



```

1 //Sensitive information must not be output by
2     // Log.e()/w()/i()
3     Log.e(LOG_TAG, "Not sensitive information (ERROR)");
4     Log.w(LOG_TAG, "Not sensitive information (WARN)");
5     Log.i(LOG_TAG, "Not sensitive information (INFO)");
6
7 //Sensitive information should be output by
8     // Log.d()/v() in case of need.
9     Log.d(LOG_TAG, "sensitive information (DEBUG)");
10    Log.v(LOG_TAG, "sensitive information (VERBOSE)");
  
```

Outputting Log to LogCat:

- If the application is for release, those two methods would be deleted automatically.
- ProGuard is used to automatically delete code blocks where **Log.d()** and **Log.v()** is called.



```

1 proguard-project.txt
2
3 -assumenosideeffects class android.util.log {
4     public static *** d(...);
5     public static *** v(...);
6 }
  
```

Outputting Log to LogCat:

- By specifying **Log.d()** and **Log.v()** as parameter of **-assumenosideeffects** option, call for **Log.d()** and **Log.v()** are granted as unnecessary code, and those are to be deleted.

```

1 proguard-project.txt
2
3 -assumenosideeffects class android.util.log {
4     public static *** d(...);
5     public static *** v(...);
6 }
  
```

Outputting Log to LogCat:

Development version application (Debug build)			Release version application (Release build)		
LogCat Search Console Search for messages. Accepts Java regexes. Prefix with pid:, app:, tag			LogCat Search Console Search for messages. Accepts Java regexes. Prefix with pid:, app:, tag		
Level	Tag	Text	Level	Tag	Text
E	ProGuardActivity	Not sensitive information (ERROR)	E	ProGuardActivity	Not sensitive information (ERROR)
W	ProGuardActivity	Not sensitive information (WARN)	W	ProGuardActivity	Not sensitive information (WARN)
I	ProGuardActivity	Not sensitive information (INFO)	I	ProGuardActivity	Not sensitive information (INFO)
D	ProGuardActivity	sensitive information (DEBUG)			
V	ProGuardActivity	sensitive information (VERBOSE)			

- Difference of LogCat output between development version application and release version application.

Detect Instrumentation tools like Frida

- Frida is a dynamic instrumentation tool used to manipulate and analyze apps at runtime.
- It allows attackers to bypass security measures, modify app behavior, and steal sensitive data.
- **Detecting and preventing Frida usage is crucial for securing applications against runtime manipulation and protecting sensitive information.**

Detect Instrumentation tools like Frida

- **Checking Frida Files**

```

1 private boolean checkFridaFiles() {
2     String[] paths = {"/data/local/tmp/frida-server", "/data/local/tmp/re.frida.server"};
3     for (String path : paths) {
4         if (new File(path).exists()) return true; // Frida file found
5     }
6     return false; // No Frida files found
7 }
  
```

- This method checks common paths for the presence of Frida-related files, which could indicate the tool is being used.

Detect Instrumentation tools like Frida

- **Checking for Frida Classes**

```
1 private boolean checkFridaClasses() {
2     try {
3         Class.forName("re.frida.Gadget"); // Try loading the class
4         return true; // Class exists (Frida detected)
5     } catch (ClassNotFoundException e) {
6         return false; // Class not found
7     }
8 }
```

- This method attempts to load the `frida.Gadget` class, which is a known part of the Frida toolkit, to detect if Frida is running.

Detect Instrumentation tools like Frida

- Scanning Local TCP Ports for Frida

```

1 private boolean checkFridaServerPort() {
2     int fridaPort = 27042; // Default Frida server port
3     return isPortOccupied(fridaPort); // Check if the port is open
4 }
5
6 private boolean isPortOccupied(int port) {
7     try (Socket socket = new Socket()) {
8         socket.connect(new InetSocketAddress("127.0.0.1", port), 1000); // Try to connect to localhost on the port
9         return true; // Port is open, indicating Frida is likely running
10    } catch (IOException ignored) { }
11    return false; // Port is closed, Frida is not running on this port
12 }

```

- This method scans open TCP ports for communication that may indicate Frida is listening for connections.

Detect Instrumentation tools like Frida

- Scanning for Frida-related Strings in Memory Sections

```

1 private boolean scanTextSectionsForStrings() {
2     try (RandomAccessFile maps = new RandomAccessFile("/proc/self/maps", "r")) {
3         String line;
4         while ((line = maps.readLine()) != null) {
5             if (line.contains(".so") && (line.contains("frida-gadget") || line.contains("frida-
agent"))) return true; // Frida found
6         }
7     } catch (IOException e) { }
8     return false; // No Frida strings found
9 }

```

- This method scans memory sections for known Frida-related strings, which helps identify if Frida libraries are loaded into the app's memory.

08

Leverage Security Frameworks and Libraries

Securing SharedPreferences data with Encrypted SharedPreferences:

- Android provides a simple framework for app developers to persistently store key-value pairs of primitive data types.
- Thanks to the **AndroidX Security Library** which was recently added.
- It is a library that wraps the default Android SharedPreferences to encrypt the key-value pairs for protecting them against attackers.

Securing SharedPreferences data with Encrypted SharedPreferences:

1. Simply just create or fetch a Master Key from the Android Keystore:



```

1 MasterKey masterKey = new MasterKey.Builder(context, MasterKey.DEFAULT_MASTER_KEY_ALIAS)
2                       .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
3                       .build();
  
```

- We're given a default key generation specification, **AES256_GCM**, to use for creating the master key and is recommended to use this specification.

Securing SharedPreferences data with Encrypted SharedPreferences:

- Now create an instance of **EncryptedSharedPreferences**, which is a wrapper around *SharedPreferences* and handles all of the encryption.

```

1 SharedPreferences sharedPreferences = EncryptedSharedPreferences.create(
2     this,
3     "secret_secure_preferences_file",
4     masterKey,
5     EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
6     EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM);
  
```

Securing SharedPreferences data with Encrypted SharedPreferences:

- Once we've created our **EncryptedSharedPreferences** instance, we can use it just like *SharedPreferences* to store and read values.



```

1 //Storing a value
2 sharedPreferences.edit()
3     .putString("key", value)
4     .apply()
5
6 //Reading a value
7 sharedPreferences.getString("key", "defaultValue");
  
```

Securing SharedPreferences data with Encrypted SharedPreferences:

- A standard SharedPreferences XML file:



```

1 <?xml version='1.0' encoding='utf-8' standalone='yes' ?>
2 <map>
3 <int name="timeout " value="500" />
4 <boolean name="is_logged_in" value="true" />
5 <string name="cdac">secure coding practices</string>
6 </map>
  
```

Securing SharedPreferences data with Encrypted SharedPreferences:

- A SharedPreferences XML file using **Encrypted SharedPreferences**:

```

1 <?xml version='1.0' encoding='utf-8' standalone='yes' ?>
2 <map>
3 <string name="MIIEpQIBAAKCAQEAYb6BkBms39I7imXMO0UW1EDJsbGNs">
4 HhiXTk3JRgAMuK0wosHLLfaVvRUuT3ICK
5 </string>
6 <string name="TuwbBU0IrAyL9znGBJ87uEi7pW0FwYwX8SZiiKnD2VZ7">
7 va6l7hf5imdM+P3KA3Jk50ZwFj1/Ed2
8 </string>
9 <string name="8lqCQqn73Uo84Rj">
10 k73tlfVNYSpshll19ztma7U">tEcsr41t5orGWT9/pqJrMC5x503cc=
11 </string>
12 </map>
  
```

Encrypting a database with SQLCipher:

- SQLCipher is one of the simplest ways to enable secure storage in an Android app, and it's compatible for devices running Android 2.1+.
- SQLCipher uses 256-bit AES in CBC mode to encrypt each database page; in addition, each page has its own random initialization vector to further increase security.
- SQLCipher is a separate implementation of the SQLite database, and rather than implementing its own encryption, it uses the widely used and tested OpenSSL libcrypto library.

Encrypting a database with SQLCipher:

1. There are several ways to handle SQLite database, either by working directly with the **SQLiteDatabase** object or using **SQLiteOpenHelper**.

But generally, if you are already using an SQLite database in your app, simply replace the **import android.database.sqlite.*** statement with **import net.sqlcipher.database.***

Encrypting a database with SQLCipher:

- The simplest way to create an encrypted SQLCipher database is to call **openOrCreateDatabase(...)** with a password:

```

1 private static final int DB_VERSION = 1;
2 private static final String DB_NAME = "my_encrypted_data.db";
3
4 public void initDB(Context context, String password) {
5     SQLiteDatabase.loadLibs(context);
6     SQLiteDatabase database = SQLiteDatabase.
7         openOrCreateDatabase(DB_NAME, password, null);
8
9     database.execSQL("create table MyTable(a, b)");
10 }
  
```

Android KeyStore Provider:

- In Android 4.3, a new facility was added to allow apps to save private encryption keys in a system **KeyStore**.
- Called **Android KeyStore**, which restricts access only to the app that created them, and it was secured using the device pin code.
- Specifically, the Android KeyStore is a certificate store, and so only public/private keys can be stored.

Android KeyStore Provider:

1. Create a handle on your app's KeyStore:

```

1 public static final String ANDROID_KEYSTORE = "AndroidKeyStore";
2
3 public void loadKeyStore() {
4     try {
5         keyStore = KeyStore.getInstance(ANDROID_KEYSTORE);
6         keyStore.load(null);
7     } catch (Exception e) {
8         // TODO: Handle this appropriately in your app
9         e.printStackTrace();
10    }
11 }
  
```

Android KeyStore Provider:

2. Generate and save the app's key pair:

```

1 public void generateNewKeyPair(String alias, Context context) throws Exception {
2     Calendar start = Calendar.getInstance();
3     Calendar end = Calendar.getInstance();
4     // expires 1 year from today
5     end.add(1, Calendar.YEAR);
6
7     KeyPairGeneratorSpec spec = new KeyPairGeneratorSpec.
8         Builder(context)
9         .setAlias(alias)
10        .setSubject(new X500Principal("CN=" + alias))
11        .setSerialNumber(BigInteger.TEN)
12        .setStartDate(start.getTime())
13        .setEndDate(end.getTime())
14        .build();
15
16    // use the Android keystore
17    KeyPairGenerator gen = KeyPairGenerator.getInstance("RSA", ANDROID_KEYSTORE);
18    gen.initialize(spec);
19
20    // generates the keypair
21    gen.generateKeyPair();
22 }
  
```

Android KeyStore Provider:

3. Retrieve the key with a given alias:



```

1 public PrivateKey loadPrivateKey(String alias) throws Exception {
2     if (keyStore.isKeyEntry(alias)) {
3         Log.e(TAG, "Could not find key alias: " + alias);
4         return null;
5     }
6     KeyStore.Entry entry = keyStore.getEntry(KEY_ALIAS, null);
7     if (!(entry instanceof KeyStore.PrivateKeyEntry)) {
8         Log.e(TAG, " alias: " + alias + " is not a PrivateKey");
9         return null;
10    }
11    return ((KeyStore.PrivateKeyEntry) entry).getPrivateKey();
12 }
  
```

Generating a Symmetric Encryption Key:

- A symmetric key describes a key that is used for both encryption and decryption.
- To create cryptographically secure encryption keys in general, we use securely generated **pseudorandom** numbers.
- **AES** is the preferred encryption standard to DES, and typically used with key sizes 128 bit and 256 bit.

Generating a Symmetric Encryption Key:

1. Write the following function to generate a symmetric AES encryption key:



```

1 public static SecretKey generateAESKey(int keysize) throws NoSuchAlgorithmException {
2   final SecureRandom random = new SecureRandom();
3   final KeyGenerator generator = KeyGenerator.getInstance("AES");
4   generator.init(keysize, random);
5   return generator.generateKey();
6 }
  
```

Generating a Symmetric Encryption Key:

2. Create a random 32-byte Initialization Vector (IV) that matches the AES key size of 256 bit.

```

1 private static IvParameterSpec iv;
2
3 public static IvParameterSpec getIV() {
4   if (iv == null) {
5     byte[] ivByteArray = new byte[32];
6     // populate the array with random bytes
7     new SecureRandom().nextBytes(ivByteArray);
8     iv = new IvParameterSpec(ivByteArray);
9   }
10  return iv;
11 }
  
```

Generating a Symmetric Encryption Key:

- Write the following function to encrypt an arbitrary string.



```

1 public static byte[] encrypt(String plainText) throws GeneralSecurityException, IOException {
2     final Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
3     cipher.init(Cipher.ENCRYPT_MODE, getKey(), getIV());
4     return cipher.doFinal(plainText.getBytes("UTF-8"));
5 }
6
7 public static SecretKey getKey() throws NoSuchAlgorithmException {
8     if (key == null) {
9         key = generateAESKey(256);
10    }
11    return key;
12 }
  
```

Generating a Symmetric Encryption Key:

- For completeness, the preceding snippet shows how to decrypt. The only difference is that we call the **Cipher.init()** method using the **Cipher.DECRYPT_MODE** constant:



```

1 public static String decrpyt(byte[] cipherText) throws GeneralSecurityException, IOException {
2     final Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
3     cipher.init(Cipher.DECRYPT_MODE, getKey(), getIV());
4     return cipher.doFinal(cipherText).toString();
5 }
  
```

Obfuscating Source Code:

- Obfuscation is the process of making code difficult to read and understand to protect it from reverse engineering.
- ProGuard, a tool included in Android's SDK, can be used to shrink, optimize, and obfuscate your application's code.

Before Rename Obfuscation:

```

1 private void
2 CalcPayroll (SpecialList employee-
3 Group) {
4     while(employeeGroup.HasMore()) {
5         employee =
6         employeeGroup.GetNext(true);
7         employee.UpdateSalary();
8         DistributeCheck(employee);
9     }
10 }
  
```

After Rename Obfuscation:

```

1 private void a(a b) {
2     while (b.a()) {
3         a = b.a(true);
4         a.a();
5         a(a);
6     }
7 }
8
9
10
  
```

09

Monitor Error and Exception Handling

- Exceptions occur when an application gets into an abnormal or error state.
- Monitor Error and Exception Handling is about ensuring that the app will handle an exception and transition to a safe state without exposing sensitive information via the UI or the app's logging mechanisms.

- Plan for standard **Runtime Exceptions** (e.g. *NullPointerException*, *IndexOutOfBoundsException*, *ActivityNotFoundException*, *CancellationException*, *SQLException*) by creating proper null checks, bound checks, and the like.
- Make sure that all confidential information handled by high-risk applications is always wiped during execution of the finally blocks.



```

1 byte[] secret;
2 try {
3     //use secret
4 } catch (SPECIFICEXCEPTIONCLASS | SPECIFICEXCEPTIONCLASS2 e) {
5     // handle any issues
6 } finally {
7     //clean the secret.
8 }
    
```

- Adding a general exception handler for uncaught exceptions is a best practice for resetting the application's state when a crash is imminent:

```

1 public class MemoryCleanerOnCrash implements Thread.UncaughtExceptionHandler {
2
3     private static final MemoryCleanerOnCrash S_INSTANCE = new MemoryCleanerOnCrash();
4     private final List<Thread.UncaughtExceptionHandler> mHandlers = new ArrayList<>();
5
6     //initialize the handler and set it as the default exception handler
7     public static void init() {
8         S_INSTANCE.mHandlers.add(Thread.getDefaultUncaughtExceptionHandler());
9         Thread.setDefaultUncaughtExceptionHandler(S_INSTANCE);
10    }
11
12    //make sure that you can still add exception handlers on top of it (required for ACRA for instance)
13    public void subscribeCrashHandler(Thread.UncaughtExceptionHandler handler) {
14        mHandlers.add(handler);
15    }
16
17    @Override
18    public void uncaughtException(Thread thread, Throwable ex) {
19
20        //handle the cleanup here
21        //....
22        //and then show a message to the user if possible given the context
23
24        for (Thread.UncaughtExceptionHandler handler : mHandlers) {
25            handler.uncaughtException(thread, ex);
26        }
27    }
28 }

```

- Now the handler's initializer must be called in your custom **Application** class (e.g., the class that extends Application):

```

1 @Override
2 protected void attachBaseContext(Context base) {
3     super.attachBaseContext(base);
4     MemoryCleanerOnCrash.init();
5 }
  
```

10

Handling Intents

- Using an **Untrusted** Intent to launch a component, for example, by calling **startActivity()**; or to return data, for example, by calling **setResult()**; is dangerous and can allow malicious apps to cause the following problems:
 - Steal sensitive files or system data (like SMS messages) from your app.
 - Launch your app's private components with **poisoned** arguments.
- It is important that your app does not call **startActivity**, **startService**, **sendBroadcast**, or **setResult** on untrusted Intents without validating or sanitizing these intents.

- Some apps contain an **Intent Redirection** issue which can allow malicious apps to access private app components or files.
- **Intent Redirection** occurs when an activity can forward intents to arbitrary components allowing them to reach even unintended private and sensitive ones.
- Usually, the redirecting activity is intended to launch only a single or a predefined set of activities.
- Limiting the components that can be reached only to those meant by the developer can eliminate the problem.

- Generally you can put an Intent to an Intent with:



```

1 Intent intent = new Intent();
2 Intent.putExtra(Intent.EXTRA_INTENT, new Intent());
  
```

- To retrieve the Intent (from within an Activity) from the intent you can do the following:



```

1 Intent intent = getIntent().getParcelableExtra(Intent.EXTRA_INTENT);
  
```

Private vs Public components can make the difference:

- If the app component does not need to receive intents from other apps then you can make that app component private by setting the **android:exported="false"** in your app's AndroidManifest.xml file



```
<[component name] android:exported="false">  
</[component name]>
```

Validating the source of the Intent:

- You can verify that the originating Activity can be trusted using methods like **getCallingActivity()**

```

1 //Check if the originating Activity is from trusted package
2 if (getCallingActivity().getPackageName().equals("desired_package")) {
3
4     Intent intent = getIntent();
5
6     //extract the nested Intent
7     Intent forward = (Intent) intent.getParcelableExtra("extra_intent");
8
9     //redirect the nested Intent
10    startActivity(forward);
11 }
    
```

Make sure that embedded intent is not harmful:

- You should verify that the redirected Intent
 - Will not be sent to any of your app's private components, and
 - Will not be sent to an external app's components

Make sure that embedded intent is not harmful:

- Apps can check which component will be used to handle the intent before redirecting it using methods like **resolveActivity()**

```

1 Intent intent = getIntent();
2
3 //extract the nested Intent
4 Intent forward = (Intent) intent.getParcelableExtra("extra_intent");
5
6 //get component name
7 ComponentName name = forward.resolveActivity(getPackageManager());
8
9 //Check that the package name and class name are as intended
10 if (name.getPackageName().equals("desired_package") &&
11     name.getClassName().equals("desired_class")) {
12     //redirect the nested Intent
13     startActivity(forward);
14 }

```

11

SSL Pinning

- SSL Pinning (Certificate Pinning) is a security technique that ties a mobile app to a specific server SSL certificate or public key.
- By default, Android and iOS apps trust any valid certificate issued by a trusted Certificate Authority (CA) — opening the door to Man-in-the-Middle (MitM) attacks.
 - Attackers can install a rogue CA on the device and intercept encrypted HTTPS traffic.
 - Tools such as Burp Suite, Charles Proxy, and mitmproxy exploit this by default.

Types of SSL Pinning:

- Certificate Pinning
 - The app stores the full server certificate (.cer / .pem) and compares it byte-for-byte during the TLS handshake.
- Public Key Pinning
 - The app pins only the server public key (as a SHA-256 Base64 hash). The key remains the same across certificate renewals.

Method 1: Network Security Config (Android 7.0+ — No Code Required):

- Android 7.0+ supports SSL pinning declaratively via res/xml/network_security_config.xml — no third-party lib
- Reference the config file in AndroidManifest.xml using the android:networkSecurityConfig attribute.
- Backup pins can be listed for failover; pin-sets support an expiration date for planned key rotation.

```

<!-- res/xml/network_security_config.xml -->
<network-security-config>
  <domain-config>
    <domain includeSubdomains="true">example.com</domain>
    <pin-set expiration="2027-01-01">
      <!-- Primary pin (public key SHA-256) -->
      <pin digest="SHA-256">7HIpactkIAq2Y49orFOOQKurWxmmSFZhBCoQYcRhJ38=</pin>
      <!-- Backup pin -->
      <pin digest="SHA-256">fwza0LRMXouZHRC8Ei+4PyulDPDcf3UKgO/04cDM1oE=</pin>
    </pin-set>
  </domain-config>
</network-security-config>

```

Method 2: OkHttp CertificatePinner (Recommended for Android):

- OkHttp (widely used by Retrofit and other HTTP stacks) provides a built-in CertificatePinner API.
- Pin the sha256/ public key hash of the leaf certificate or any CA in the chain.
- Multiple pins for the same hostname can be listed — ideal for zero-downtime key rotation.

```
<rtificatePinner pinner = new CertificatePinner.Builder()
    .add("example.com",
        "sha256/7HIpactkIAq2Y49orFOOQKurWxmmSFZhBCoQYcRhJ38=",
        "sha256/fwza0LRMXouZHRC8Ei+4PyuldPDcf3UKgO/04cDM1oE=")
    .build();

OkHttpClient client = new OkHttpClient.Builder()
    .certificatePinner(pinner)
    .connectTimeout(30, TimeUnit.SECONDS)
    .build();
```

Method 3: Custom TrustManager — Native Android / Java:

- For apps not using OkHttp, a custom X509TrustManager can be implemented to validate the certificate manually against a bundled asset.
- The certificate (.cer) is placed in res/raw/ or assets/ and loaded into a KeyStore at runtime.
- Warning: A TrustManager that blindly accepts all certificates is WORSE than no pinning — it creates a false sense of security.



```
keyStore keyStore = KeyStore.getInstance(KeyStore.getDefaultType());
InputStream certInput = context.getResources().openRawResource(R.raw.my_cert);
Certificate ca = CertificateFactory.getInstance("X.509").generateCertificate(certInput);
keyStore.setCertificateEntry("ca", ca);

TrustManagerFactory tmf = TrustManagerFactory
    .getInstance(TrustManagerFactory.getDefaultAlgorithm());
tmf.init(keyStore);

SSLContext sslContext = SSLContext.getInstance("TLS");
sslContext.init(null, tmf.getTrustManagers(), null);
// Attach sslContext to your HttpURLConnection or HttpClient
```

Extracting the SHA-256 Public Key Hash (OpenSSL commands):

- Before pinning you must extract the server certificate public key hash using OpenSSL or similar tools.
- Pin both the leaf certificate and at least one intermediate CA for resilience — use backup pins.
- OkHttp also prints the correct pin values in its `SSLPeerUnverifiedException` message when a connection fails — useful for first-time setup.



```
# Step 1: Fetch the server certificate
openssl s_client -connect example.com:443 -servername example.com </dev/null \
| openssl x509 -outform DER -out server.der

# Step 2: Extract the public key and produce the SHA-256 Base64 hash
openssl x509 -inform DER -in server.der -pubkey -noout \
| openssl pkey -pubin -outform DER \
| openssl dgst -sha256 -binary \
| openssl enc -base64

# Output (use this as the pin value):
# 7HIpactkIAq2Y49orFOOQKurWxmmSFZhBCoQYcRhJ38=
```

SSL Pinning — Best Practices:

DO

- + Pin the public key hash (not the full cert)
- + Always include at least one backup pin
- + Set an expiration date on your pin-set
- + Test on both debug and release builds
- + Combine with root / jailbreak detection
- + Use report-only mode during initial rollout
- + Store validation logic in native (NDK) code
- + Have a documented key rotation plan

DON'T

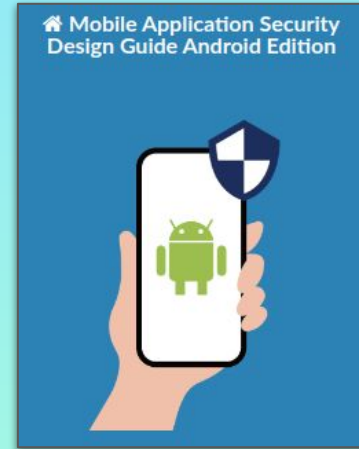
- Pin the full certificate (breaks on renewal)
- Accept all certs in a custom TrustManager
- Hard-code only one pin with no backup
- Disable pinning in production builds
- Skip re-testing after certificate renewal
- Rely on pinning as the sole security layer
- Ship the private key or full cert chain in APK

Conclusion

- Security is not a single measure but a combination of multiple practices working together.
- It follows the concept of **defense in depth**, where multiple layers act collectively to safeguard the application.
- Ensuring robust security means implementing various measures in place to protect against diverse threats.

Additionally, maintaining robust security requires ongoing vigilance through regular updates, security reviews, and adapting to evolving threats.

Android Application Secure Design/Secure Coding Guidebook



Mobile Application Security Design Guide Android Edition

Thanks!

Do you have any questions?